

Computer Hardware

Deadlock and Cooperating Threads

Ksawery Moźdzynski



WORK IN PROGRESS

Warning: this presentation is still under construction.
It has not been run and tested yet.



Agenda For Today



1. Deadlock

Why the bank stopped, and five hungry philosophers



2. Fixing it

A fixed lock order, `std::scoped_lock`



3. Waiting

Condition variables: no busy waiting



4. Producer and consumer

Two threads and a buffer



5. Semaphores

Up to N threads at the same time



6. A thread pool

Your own small scheduler

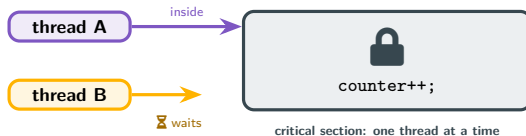
From Tuesday: threads and the race

Threads

A process has its own memory and one or more threads. The threads **share** the memory; each one has its own stack and registers. `std::thread` starts a thread, `join` waits for it.

⚠ Race condition

Two threads change shared data at the same time. The result depends on **when** each one runs, and some updates are lost.



A **critical section**: code that only **one thread at a time** may run.

From Tuesday: making it correct

Mutex

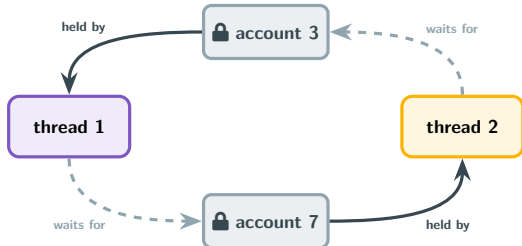
lock: one thread goes in, the others are **Blocked** and use no CPU. `std::lock_guard` unlocks at the end of the block, even after an early return. On the ESP32 the same: `xSemaphoreTake`, `xSemaphoreGive`.

Atomics and sharing less

`std::atomic`: the CPU does read, add, write as **one** step. Local sum: each thread works on its own data, and the results are put together once, at the end.

Being correct costs time: every lock is a small queue. Share less, lock less.

From Tuesday: lock granularity and deadlock



One big lock

Simple and correct, but only one thread at a time: a queue.

Many small locks

More threads work at the same time. But a thread may need **two** locks...

⚠ Deadlock

Threads wait for each other **forever**. The program hangs, and the CPU usage drops to 0%.

The question

How can a thread lock **two** accounts
without a deadlock?

Today: the answer. Then: threads that do not only stay out of each other's way, but **work together**.

Task 0: one more time

Task

Run your `bank.cpp` from step 3 (or the one from [GitHub](#)) three times, with 4 threads.

1. How many runs finish?
2. While it hangs: what does Task Manager, Activity Monitor or `top` show for the CPU usage?

Solution: one more time

Question 1

None: it never finishes.

Question 2

About 0%. The threads do not work: they are **Blocked**, they wait.

A program that hangs and uses no CPU: the typical sign of a deadlock.

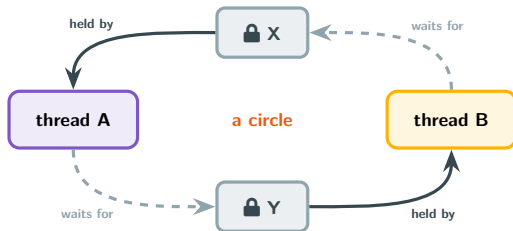
Deadlock

A **deadlock**: a group of threads where each one waits for something that **another one** in the group holds. Nobody can continue, ever.

Why nobody helps

A Blocked thread gets no time slices (Lecture 6, task states). It waits for a mutex, and the mutex is never given back.

The wait-for graph



Two kinds of arrows

From a lock to the thread that **holds** it, and from a thread to the lock it **waits for**.

A deadlock

In the graph, a deadlock is a **circle**.

Thread A holds X and waits for Y. Thread B holds Y and waits for X.

Four conditions for a deadlock

1 Mutual exclusion

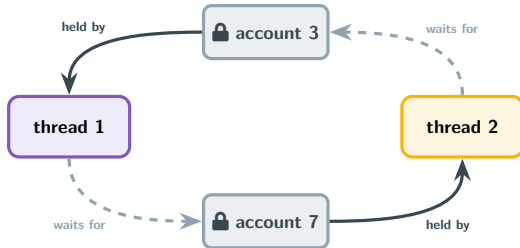
2 Hold and wait

3 No preemption

4 Circular wait

A deadlock needs **all four**. Break **one** of them, and a deadlock is impossible. One by one, in the bank:

Condition 1: mutual exclusion

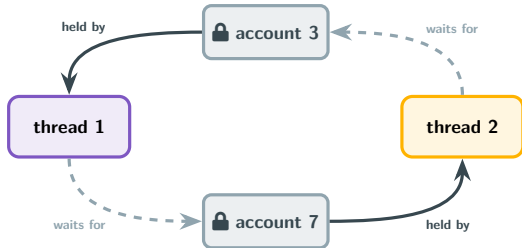


1 A lock has only **one** owner at a time.

In the bank

$m[3]$ is held by thread 1. Nobody else can have it now.

Condition 2: hold and wait

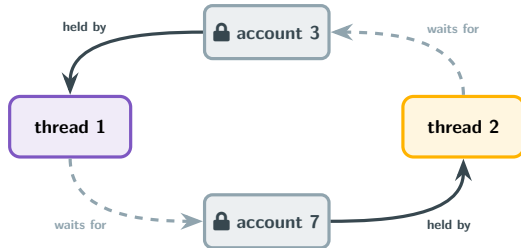


2 A thread **holds** one lock and **waits** for another.

In the bank

Thread 1 holds $m[3]$ and waits for $m[7]$.

Condition 3: no preemption

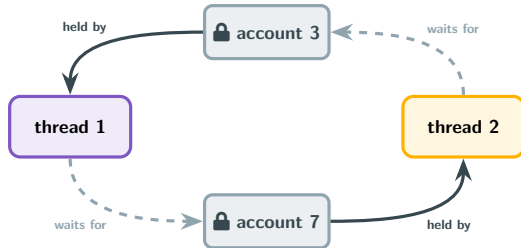


3 Nobody can take a lock **away** from a thread. Only its owner gives it back.

In the bank

Nobody can take $m[3]$ from thread 1.

Condition 4: circular wait



4 There is a **circle**: each thread waits for the next one.

In the bank

Thread 1 → $m[7]$ → thread 2 → $m[3]$ → thread 1.

Which condition can we break?

① and ③: we keep them

They are what a mutex **is**: one owner, and nobody takes it away.

② and ④: we can break them

They are about **how** the threads take their locks. That is up to us.

Next: try it in the simulator, then two ways to break ② or ④.

Simulator 10: deadlock

Simulator task

Topic **Deadlock**: thread A locks X, then Y. Thread B locks Y, then X. You choose which thread runs the next step.

1. Find an order of steps that ends in a **deadlock**. How many steps does it take?
2. Find an order that finishes **without** a deadlock.
3. Switch thread B to *X then Y*. Can you still make a deadlock?

 [ksawerym.github.io/ComputerHardwareSimulators/#cc-dead](https://github.com/ksawerym/ComputerHardwareSimulators/#cc-dead)

Solution: Simulator 10

Question 1

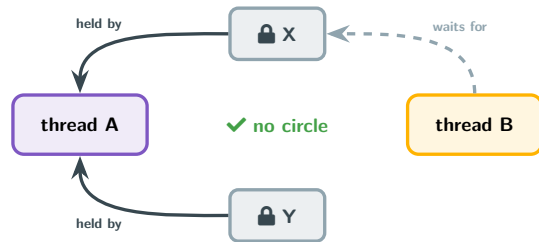
Two steps are enough: A locks X, B locks Y. Now A waits for Y and B waits for X: a circle.

Question 2

One thread takes **both** locks before the other one starts.

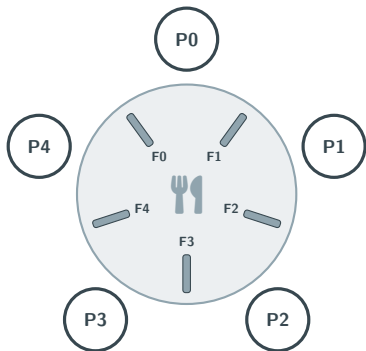
Question 3

No. Whoever gets X first also gets Y. The other one waits for X and holds **nothing**.



The same order of locks in every thread $\Rightarrow$ no circle $\Rightarrow$ no deadlock.

The dining philosophers



Five philosophers, five forks

They think, get hungry and eat. To eat, a philosopher needs **both** forks on either side. A fork is a **mutex**.

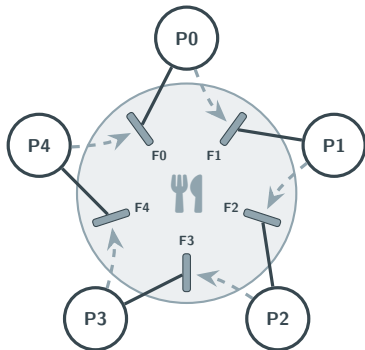
Each philosopher (not C++ yet)

```
● ● ●  
for (;;) {  
    think();  
    take the left fork;  
    take the right fork;  
    eat();  
    put both forks down;  
}
```

The question

Everybody gets hungry at the same moment. What happens?

Everybody takes the left fork



Each one holds a fork...

P0 holds F0, P1 holds F1, ..., P4 holds F4.

! ... and waits for the next one

P0 waits for F1, which P1 holds. P1 waits for F2, ..., and P4 waits for F0, which P0 holds.

— holds - - - waits for

Five philosophers, five forks, nobody eats. The same circle as in the bank, only bigger.

Task 1: the philosophers in C++

philosophers.cpp (a part)

```
● ● ●  
const int P = 5, MEALS = 1000;  
std::mutex fork_[P];  
int meals[P];  
  
void philosopher(int i) {  
    int left = i, right = (i + 1) % P;  
    for (int k = 0; k < MEALS; k++) {  
        std::lock_guard<std::mutex> a(fork_[left]);  
        // look around  
        std::lock_guard<std::mutex> b(fork_[right]);  
        meals[i]++;  
        // eat  
    }  
}  
// main(): start 5 philosophers, join, print meals
```

fork_ has an underscore because fork is already the name of a function of the operating system.

Task

Download philosophers.cpp from [GitHub](#).
Compile it with -O0.

1. Run it 5 times. Does it ever hang?
2. Replace the comment look around with a short pause:
`std::this_thread::sleep_for(
 std::chrono::microseconds(10));`
It stops this thread for 10 μ s. Run it again.

Solution: the philosophers in C++

Question 1

It depends on the machine. With 2 cores it **never** hung: the time between the two locks is so short that the circle almost never closes. With 8 cores it hung in 5 of 5 runs.

Question 2

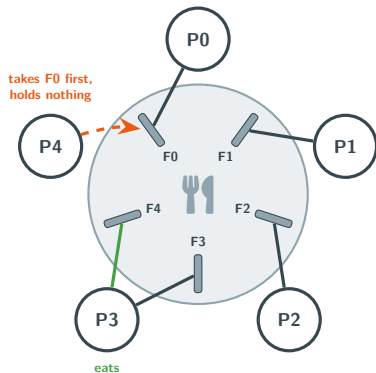
Now it hangs **every** time: all five hold the left fork while they look around.

⚠ A deadlock can hide

The program passes every test, and one day it hangs at a customer. **Testing cannot prove that there is no deadlock.**

We need a rule that makes the circle **impossible**, not just unlikely.

Rule: always lock in the same order



Give every lock a **number**. Every thread takes its locks in **increasing** order.

Why there is no circle

In a circle, the thread that holds the **highest** lock would wait for a **lower** one. The rule forbids that.

Only P4 changes: it takes F0 first. One philosopher is enough to break the circle.

Rule: always lock in the same order, in C++

```
void philosopher(int i) {  
    int left = i, right = (i + 1) % P;  
    int first = std::min(left, right);  
    int second = std::max(left, right);  
    for (int k = 0; k < MEALS; k++) {  
        std::lock_guard<std::mutex> a(fork_[first]);  
        std::lock_guard<std::mutex> b(fork_[second]);  
        meals[i]++;  
        // eat  
    }  
}
```

The lower number first

`std::min` and `std::max` give the smaller and the bigger number (`#include <algorithm>`).

P0 to P3: first is still the left fork. P4: left = 4, right = 0, so it takes F0 first.

Two locks at once: `std::scoped_lock`

```
● ● ●  
void transfers(int id) {  
    // ... rng, acc, amount as on Tuesday ...  
    for (int i = 0; i < TRANSFERS; i++) {  
        int from = acc(rng), to = acc(rng);  
        if (from == to) continue;  
        int x = amount(rng);  
        std::scoped_lock both(m[from], m[to]);  
        balance[from] -= x;  
        balance[to] += x;  
    } // both are unlocked here  
}
```

Lock several mutexes together

Like `lock_guard`, but for **several** mutexes. It locks them all **without a deadlock**, in any order you write them, and unlocks them at the end of the block.

No `<std::mutex>`: the compiler finds the types itself (C++17).

std::scoped_lock: how does it do it?

It never waits while it holds a lock

It takes one lock and only **tries** the others. If one is busy, it gives back what it has, waits for the busy one, and tries again.

So it breaks 2 hold and wait.

Which condition do we break?

Method	Breaks
Lock order (by number)	4 circular wait
<code>std::scoped_lock</code>	2 hold and wait: it never waits while it holds a lock
One big lock (Tuesday, step 2)	only one lock: a circle is impossible

Two tools: **your own order** (works everywhere, also on the ESP32), or `scoped_lock` (the simplest in C++).

Task 2: fix the philosophers and the bank

Task

Fix both programs. Both are on [GitHub](#).

1. **Philosophers** (with the pause): take the forks in increasing order. Do they always finish? Does everybody eat 1000 times?
2. **Bank, step 4**: lock the two accounts in increasing order. Is the sum always 10 000?
3. **Bank, step 5**: use `std::scoped_lock` instead.
4. Measure steps 2, 4 and 5 with 1, 2 and 4 threads.

If you finish early: set `ACCOUNTS = 1000` and add slow work inside the lock. Compare step 2 and step 4 again, with 2 threads.



```
volatile int sink;
void slowWork() {
    for (int k = 0; k < 500; k++)
        sink = k;
}
```

`volatile`, as on Monday: the compiler may not remove the loop.

Solution: fix the philosophers and the bank

Step 4



```
int first = std::min(from, to);  
int second = std::max(from, to);  
std::lock_guard<std::mutex> a(m[first]);  
std::lock_guard<std::mutex> b(m[second]);
```

Questions 1 to 3

Always finishes, everybody eats 1000 times, the sum is always 10 000.

Question 4

With 10 accounts: no real gain over one big lock.
Transfers often meet on the same account, and the work under the lock is tiny.

If you finished early

1000 accounts, longer work, 2 threads (a test machine): one big lock $\approx$ 1200 ms, a lock per account $\approx$ 470 ms.

Small locks pay off when threads **rarely** meet and the work under the lock is **long**. Always measure.

A new problem: wait until something is ready

So far: *do not disturb me* (a mutex). Now: *wake me up when it is ready*.



Busy waiting

```
while (!ready) { }
```



Sleep and check

```
while (!ready)  
sleep_for(100ms);
```



Wait until told

a condition variable

Remember polling and interrupts from Lecture 6? The same story again.

Busy waiting is polling

On Monday: polling

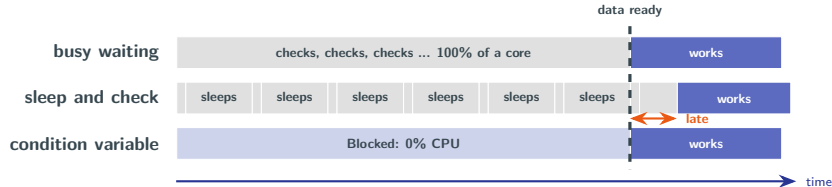
Polling is like checking your mailbox: the CPU asks the button again and again. “Check all the time: we waste the CPU.” “Check from time to time: we miss presses.”

Today: waiting for a thread

Busy waiting = check all the time: one core at 100%.
Sleep and check = check from time to time: we react late.

On Monday the answer was an **interrupt**: the button tells the CPU. Today: another thread tells you.

Three ways to wait



	CPU while waiting	Reacts
busy waiting	100% of a core	at once
sleep and check	almost 0%	up to 100 ms late
condition variable	0%	at once

A condition variable is the **interrupt** of threads: the waiting thread is Blocked until it is woken up.

A condition variable in C++

```
● ● ●  
  
#include <condition_variable>  
std::mutex m;  
std::condition_variable cv;  
bool ready = false;  
void worker() {  
    std::unique_lock<std::mutex> lock(m);  
    cv.wait(lock, [] { return ready; });  
    std::cout << "The data is ready\n";  
}  
  
int main() {  
    std::thread w(worker);  
    // ... prepare the data ...  
    {  
        std::lock_guard<std::mutex> lock(m);  
        ready = true;  
    }  
    cv.notify_one();  
    w.join();  
}
```

The whole program

The worker waits until main has prepared the data and set ready. (Also #include <iostream>, <thread> and <mutex>.)

Four new things

std::condition_variable, std::unique_lock, wait and notify_one. Let us look at them **one by one**.

A condition variable: `std::condition_variable`

```
● ● ●  
std::mutex m;  
std::condition_variable cv;  
bool ready = false;  
  
void worker() {  
    std::unique_lock<std::mutex> lock(m);  
    cv.wait(lock, [] { return ready; });  
    std::cout << "The data is ready\n";  
}
```

A place to wait

Threads can sleep on cv until another thread wakes them up.

It holds **no** data. The condition itself is ready, a normal variable protected by the mutex m. (`#include <condition_variable>`)

A condition variable: `std::unique_lock`

```
● ● ●  
std::mutex m;  
std::condition_variable cv;  
bool ready = false;  
  
void worker() {  
    std::unique_lock<std::mutex> lock(m);  
    cv.wait(lock, [] { return ready; });  
    std::cout << "The data is ready\n";  
}
```

Like `lock_guard`, but more flexible

It locks `m` and unlocks it at the end of the block, just like `lock_guard`.

The difference: it can be **unlocked and locked again** in the middle. `wait` needs exactly that.

A condition variable: wait

```
● ● ●  
std::mutex m;  
std::condition_variable cv;  
bool ready = false;  
  
void worker() {  
    std::unique_lock<std::mutex> lock(m);  
    cv.wait(lock, [] { return ready; });  
    std::cout << "The data is ready\n";  
}
```

Sleep until the condition is true

[] { return ready; }: a lambda that gives the condition.

If it is false: **unlock** m and sleep (**Blocked**). When woken up: lock m again and **check again**.

After wait: the condition is true, and we hold m.

A condition variable: `notify_one`

```
● ● ●
int main() {
    std::thread w(worker);
    // ... prepare the data ...
    {
        std::lock_guard<std::mutex> lock(m);
        ready = true;
    }
    cv.notify_one();
    w.join();
}
```

Wake up one thread

It wakes up **one** thread that sleeps on cv. That thread checks ready again, sees true, and goes on.

ready is set **under the mutex**: the worker reads it under the mutex too.

wait: why the condition?

A notify can come too early

If `main` calls `notify_one` **before** the worker waits, nobody is woken up. With the condition, the worker sees `ready = true` and does not sleep at all.

A thread can wake up by itself

The system may wake a thread **without** a notify (a *spurious wakeup*). With the condition, it checks again and goes back to sleep.

Always give wait the condition.

Task 3: three ways to wait

Task

Download `waiting.cpp` from [GitHub](#). `main` prepares the data for 3 seconds, then sets `ready`. The worker waits for it and prints how late it reacted. Version 1 is ready.

1. Run version 1. Look at the CPU usage while it waits.
2. Version 2: add `std::this_thread::sleep_for(std::chrono::milliseconds(100));` inside the loop. CPU usage? How late does the worker react?
3. Version 3: use a condition variable. CPU usage? How late does it react?

If you finish early: start 4 workers and wake them all up with one call. Which function do you need?

Solution: three ways to wait

Question 1

One core at **100%** for 3 seconds, just to check one variable.

Question 2

Almost 0%, but it reacts up to **100 ms** late.

Question 3

0%, and it reacts **at once** (well below 1 ms).

If you finished early
`cv.notify_all()`.

Do not wait in a loop. Let another thread **wake you up**: wait and notify.

Producer and consumer



A **producer** puts data into a shared **buffer**, a **consumer** takes it out. They work at different speeds.

Examples: a keyboard and a program; a network card and the network stack; a camera and a video encoder; a button ISR and the LCD task.

Producer and consumer: when do they wait?

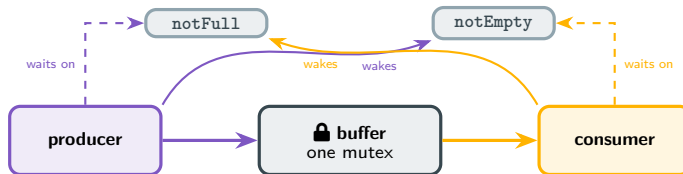


⚠ **The buffer is full**
The producer must **wait** until a place is free.

⚠ **The buffer is empty**
The consumer must **wait** until there is data.

Two things to wait for: two condition variables.

The buffer: one mutex, two conditions



	waits for	after its work, wakes up
producer	notFull	notEmpty
consumer	notEmpty	notFull

Each one waits for its own condition, and wakes up the **other** one.

Simulator 11: producer and consumer

Simulator task

Topic **Producer-consumer**. Buffer size = 5.

1. The producer is **faster** than the consumer. What happens to the buffer? In which state is the producer most of the time?
2. The consumer is faster. And now?
3. Same speed, with *random short pauses* on. Change the buffer size from **1** to **8**. What does a bigger buffer give?

 ksawerym.github.io/ComputerHardwareSimulators/#cc-pc

Solution: Simulator 11

Question 1

The buffer fills up. Then the producer is **Blocked** most of the time: it runs at the speed of the consumer.

Question 2

The buffer is almost always empty. The consumer is Blocked and waits for each item.

Question 3

A bigger buffer **absorbs** short pauses. With size 1 they wait for each other all the time.

In the long run the whole system runs at the speed of the **slower** one. The buffer only smooths out short differences.

The bounded buffer in C++



```
#include <queue>
#include <condition_variable>

const int SIZE = 5, ITEMS = 1000;
std::queue<int> buffer;
std::mutex m;
std::condition_variable notFull, notEmpty;

void producer() {
    for (int i = 1; i <= ITEMS; i++) {
        std::unique_lock<std::mutex> lock(m);
        notFull.wait(lock,
            [] { return buffer.size() < SIZE; });
        buffer.push(i);
        notEmpty.notify_one();
    }
}
```

The producer

It puts the numbers 1 to 1000 into a buffer with 5 places.

One new thing

`std::queue`. The rest you know: a mutex, wait and notify.

The bounded buffer: `std::queue`

```
● ● ●
#include <queue>
#include <condition_variable>

const int SIZE = 5, ITEMS = 1000;
std::queue<int> buffer;
std::mutex m;
std::condition_variable notFull, notEmpty;

void producer() {
    for (int i = 1; i <= ITEMS; i++) {
        std::unique_lock<std::mutex> lock(m);
        notFull.wait(lock,
            [] { return buffer.size() < SIZE; });
        buffer.push(i);
        notEmpty.notify_one();
    }
}
```

A queue: first in, first out

`push(x)` adds at the back.

`front()` reads the first one, `pop()` removes it.

`size()`, `empty()`: how many, or none?

It has no limit: the limit `SIZE` is ours.

The bounded buffer: wait and notify



```
#include <queue>
#include <condition_variable>

const int SIZE = 5, ITEMS = 1000;
std::queue<int> buffer;
std::mutex m;
std::condition_variable notFull, notEmpty;

void producer() {
    for (int i = 1; i <= ITEMS; i++) {
        std::unique_lock<std::mutex> lock(m);
        notFull.wait(lock,
            [] { return buffer.size() < SIZE; });
        buffer.push(i);
        notEmpty.notify_one();
    }
}
```

Wait for a place, wake the consumer

notFull.wait: sleep while the buffer is full.

notEmpty.notify_one: there is a new item, so wake up the consumer if it sleeps.

All of it under **one** mutex m.

Task 4: the bounded buffer

Task

Download `buffer.cpp` from [GitHub](#). It has the producer from the slide before. Write `consumer()`.

1. The consumer takes ITEMS numbers from the buffer and adds them up. Expected sum:
 $1 + 2 + \dots + 1000 = \mathbf{500\,500}$. Is your sum correct?
2. In the producer, print a message when it has to wait (the buffer is full). How often does it wait? Try `SIZE = 1` and `SIZE = 100`.
3. What happens if you forget `notFull.notify_one()` in the consumer?

If you finish early: two producers (1 ... 1000 each) and two consumers. Expected sum: 1 001 000. How does each consumer know when to stop?

Solution: the bounded buffer

```
● ● ●
long long consumer() {
    long long sum = 0;
    for (int i = 1; i <= ITEMS; i++) {
        std::unique_lock<std::mutex> lock(m);
        notEmpty.wait(lock,
            [] { return !buffer.empty(); });
        sum += buffer.front();
        buffer.pop();
        notFull.notify_one();
    }
    return sum;
}
```

Questions 1 and 2

500 500. The smaller the buffer, the more often the producer waits.

Question 3

Once the buffer is full, the producer sleeps and **nobody wakes it up**: the program hangs. No circle of locks, but the same effect: a thread waits forever.

If you finished early

Each consumer takes 1000 items. Or: at the end, the producers put a special value (e.g. -1, a *poison pill*) for each consumer.

Semaphores



A **semaphore** counts free places. `acquire()`: take a place, or wait if there is none. `release()`: give a place back and wake up one waiting thread.

A parking with 3 places: 3 cars inside, the fourth waits at the gate.

A semaphore in C++

```
● ● ●  
#include <semaphore>                                // C++20  
  
std::counting_semaphore<3> parking(3);  
  
void car(int id) {  
    parking.acquire();    // wait for a place  
    // ... park ...  
    parking.release();    // leave  
}
```

The whole program

Each car is a thread. At most 3 of them are parked at the same time.

Three new things

std::counting_semaphore, acquire and release. Let us look at them **one by one**.

A semaphore in C++: `std::counting_semaphore`

```
● ● ●  
#include <semaphore> // C++20  
  
std::counting_semaphore<3> parking(3);  
  
void car(int id) {  
    parking.acquire(); // wait for a place  
    // ... park ...  
    parking.release(); // leave  
}
```

A counter of free places

`<3>`: the counter can never be more than 3.

`(3)`: it starts at 3, so all places are free.

New in C++20, `#include <semaphore>`.

A semaphore in C++: acquire

```
● ● ●  
  
#include <semaphore>                                // C++20  
  
std::counting_semaphore<3> parking(3);  
  
void car(int id) {  
    parking.acquire();    // wait for a place  
    // ... park ...  
    parking.release();    // leave  
}
```

Take a place

If the counter is more than 0: subtract 1 and go on.

If it is 0: wait, **Blocked**, until another thread calls release().

A semaphore in C++: release

```
● ● ●  
#include <semaphore>                                // C++20  
  
std::counting_semaphore<3> parking(3);  
  
void car(int id) {  
    parking.acquire();    // wait for a place  
    // ... park ...  
    parking.release();    // leave  
}
```

Give a place back

Add 1 to the counter. If a thread waits in `acquire()`, it wakes up.

A mutex and a semaphore

A mutex

One owner. Only the thread that locked it may unlock it.

A semaphore

Up to **N** threads at the same time. **Any** thread may call `release()`, also one that did not call `acquire()`.

You saw a semaphore on Tuesday

In FreeRTOS a mutex is a kind of semaphore: `xSemaphoreCreateMutex`, `xSemaphoreTake`, `xSemaphoreGive`. A plain FreeRTOS semaphore is how an ISR wakes up a task.

The buffer with two semaphores

Producer

```
● ● ●  
std::counting_semaphore<SIZE> freeSlots(SIZE);  
std::counting_semaphore<SIZE> fullSlots(0);  
  
// for each item:  
freeSlots.acquire();           // a free place  
{  
    std::lock_guard<std::mutex> lock(m);  
    buffer.push(i);  
}  
fullSlots.release();           // one more item
```

Consumer

```
● ● ●  
// for each item:  
fullSlots.acquire();           // an item  
{  
    std::lock_guard<std::mutex> lock(m);  
    sum += buffer.front();  
    buffer.pop();  
}  
freeSlots.release();           // one more place
```

The semaphores count the places and the items. The mutex protects the queue itself.

Task 5: a semaphore

Task

A print room with 2 printers. Template: [GitHub](#).

1. Start **8** threads. Each one prints a document: `sleep_for(500ms)`. At most **2** threads may print at the same time: use a `std::counting_semaphore`.
2. Count in a `std::atomic<int>` inside how many threads are printing right now, and print it. Is it ever more than 2?
3. How long does the whole program take? **Calculate first**, then measure.

If you finish early: change Task 4 to the version with two semaphores. Is the sum still 500 500?

Solution: a semaphore



```
std::counting_semaphore<2> printers(2);
std::atomic<int> inside{0};

void print(int id) {
    printers.acquire();
    int now = ++inside;
    std::cout << id << " prints, inside: "
              << now << "\n";
    std::this_thread::sleep_for(
        std::chrono::milliseconds(500));
    --inside;
    printers.release();
}
```

Question 2

Never more than 2.

Question 3

8 documents, 2 at a time, 0.5 s each: 4 rounds, about 2 s.

If you finished early

Yes, 500 500.

A thread pool



A **thread pool**: a fixed group of threads (*workers*) that take tasks from a shared queue.

On Monday the scheduler gave tasks to **one core**. A thread pool gives tasks to **threads**: your own small scheduler.

A thread pool: why?

Threads are expensive

Starting a thread costs time. Thousands of small tasks do not need thousands of threads, only as many as there are **cores**.

Nothing new inside

A producer and consumers: main produces tasks, the workers consume them. A queue, a mutex and a condition variable.

The pool in C++

```
● ● ●  
class ThreadPool {  
public:  
    explicit ThreadPool(int n) {  
        for (int i = 0; i < n; i++)  
            workers.emplace_back([this] { work(); });  
    }  
    void submit(std::function<void()> task) {  
        { std::lock_guard<std::mutex> lock(m);  
          tasks.push(task); }  
        cv.notify_one();  
    }  
    ~ThreadPool();    // TODO: stop and join  
private:  
    void work();      // TODO: take a task, run it  
    std::vector<std::thread> workers;  
    std::queue<std::function<void()>> tasks;  
    std::mutex m;  
    std::condition_variable cv;  
    bool stop = false;  
};
```

The whole class

The skeleton of the pool. You write the two TODO functions in Task 6.

Three new things

std::function, the lambda [this], and the destructor ~ThreadPool. Let us look at them one by one.

The pool in C++: `std::function`

```
● ● ●
class ThreadPool {
public:
    explicit ThreadPool(int n) {
        for (int i = 0; i < n; i++)
            workers.emplace_back([this] { work(); });
    }
    void submit(std::function<void()> task) {
        { std::lock_guard<std::mutex> lock(m);
          tasks.push(task); }
        cv.notify_one();
    }
    ~ThreadPool();    // TODO: stop and join
private:
    void work();      // TODO: take a task, run it
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;
    std::mutex m;
    std::condition_variable cv;
    bool stop = false;
};
```

A task

`std::function<void()>`: **any** function without arguments that returns nothing, for example a lambda.

The queue tasks holds such functions. A worker takes one and calls it: `task()`. (`#include <functional>`)

The pool in C++: the lambda [this]

```
● ● ●
class ThreadPool {
public:
    explicit ThreadPool(int n) {
        for (int i = 0; i < n; i++)
            workers.emplace_back([this] { work(); });
    }
    void submit(std::function<void()> task) {
        { std::lock_guard<std::mutex> lock(m);
          tasks.push(task); }
        cv.notify_one();
    }
    ~ThreadPool();    // TODO: stop and join
private:
    void work();      // TODO: take a task, run it
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;
    std::mutex m;
    std::condition_variable cv;
    bool stop = false;
};
```

Each worker runs work()

A lambda, as in hello.cpp. Between []: what it takes from outside. this: this pool, so it can call work() of this pool.

n workers start in the constructor, all running work().

The pool in C++: the destructor

```
class ThreadPool {  
public:  
    explicit ThreadPool(int n) {  
        for (int i = 0; i < n; i++)  
            workers.emplace_back([this] { work(); });  
    }  
    void submit(std::function<void()> task) {  
        { std::lock_guard<std::mutex> lock(m);  
          tasks.push(task); }  
        cv.notify_one();  
    }  
    ~ThreadPool();    // TODO: stop and join  
private:  
    void work();      // TODO: take a task, run it  
    std::vector<std::thread> workers;  
    std::queue<std::function<void()>> tasks;  
    std::mutex m;  
    std::condition_variable cv;  
    bool stop = false;  
};
```

When the pool ends

The destructor runs when the pool goes out of scope, as with `lock_guard` on Tuesday.

Our pool's destructor must stop the workers and **join** them, after they have done all tasks.

Task 6: count primes with a pool

main() in pool.cpp

```
● ● ●  
const int LIMIT = 5'000'000, CHUNK = 100'000;  
std::atomic<int> primes{0};  
{  
    ThreadPool pool(WORKERS);  
    for (int start = 0; start < LIMIT;  
         start += CHUNK)  
        pool.submit([start, &primes] {  
            int c = 0;  
            for (int n = start;  
                 n < start + CHUNK; n++)  
                if (isPrime(n)) c++;  
            primes += c;  
        });  
} // the destructor waits for all tasks
```

[start, &primes]: the lambda takes a **copy** of start and a **reference** to primes.

Task

Download pool.cpp from [GitHub](#). Write work() and the destructor. Compile with -O2.

1. work(): wait for a task **or** for stop; take the task, **unlock**, run it.
2. The destructor: set stop, wake up **all** workers, join them.
3. Measure with 1, 2, 4 and hardware_concurrency() workers. Expected: **348 513** primes.

If you finish early: why must the task run **after** the unlock?

Solution: count primes with a pool

```
● ● ●  
void ThreadPool::work() {  
    while (true) {  
        std::function<void()> task;  
        {  
            std::unique_lock<std::mutex> lock(m);  
            cv.wait(lock, [this] {  
                return stop || !tasks.empty(); });  
            if (stop && tasks.empty()) return;  
            task = tasks.front();  
            tasks.pop();  
        }  
        task();  
    }  
}  
ThreadPool::~ThreadPool() {  
    { std::lock_guard<std::mutex> lock(m);  
      stop = true; }  
    cv.notify_all();  
    for (auto &w : workers) w.join();  
}
```

Time (a test machine, 2 cores)

Workers	Time
1	2050 ms
2	960 ms
4	1020 ms

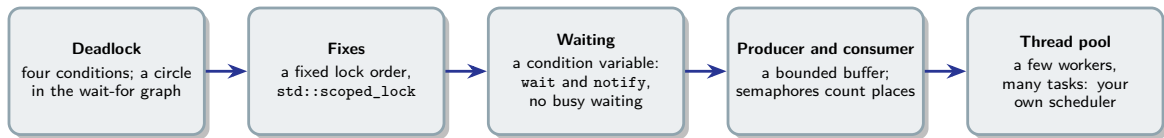
Question 3

Up to the number of cores: $2\times$ faster on 2 cores. More workers than cores: no gain.

If you finished early

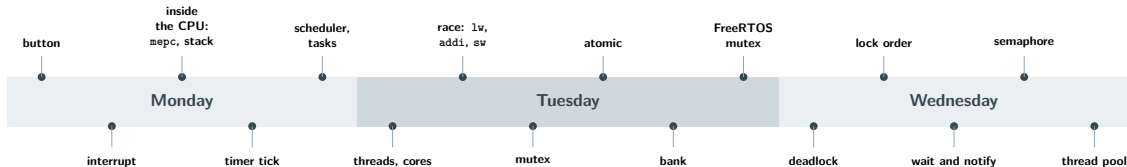
Under the lock, the other workers could not take tasks: they would run one after another again.

What we saw today



Threads must not only stay out of each other's way. They must also **work together**.

Three days: from a button to a thread pool



A few ideas, from a small microcontroller to a laptop with many cores.

Three days: the same ideas, twice

Waiting

polling ↔ busy waiting
an interrupt ↔ notify
a Blocked task in FreeRTOS ↔ a thread in wait

Sharing and scheduling

presses++ in an ISR ↔ counter++ in a thread
xSemaphoreTake ↔ std::mutex
the scheduler ↔ a thread pool

Mini-project (if there is time)

1 The auditor
C++, the bank.

2 A pipeline
C++, two bounded buffers.

3 Back to the ESP32
Wokwi, a FreeRTOS queue.

Choose one. Each one has a template on GitHub.

Mini-project 1: the auditor

The task

Today's bank (step 4, the lock order), plus one more thread, the **auditor**. While the transfers run, it checks the sum **every 100 ms** and prints it. It must lock **all** accounts, without a deadlock. Template: [GitHub](#).

Hint

The lock order: the auditor takes $m[0]$, $m[1]$, $\dots$, $m[9]$, in this order. Why can it not deadlock with the transfers?

Mini-project 2: a pipeline

The task

Three threads connected by two bounded buffers:

generate the numbers 1 to 1000 → square them → add them up.

Template: [GitHub](#).

Check the result

$1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$. For $n = 1000$: **333 833 500**.

Mini-project 3: back to the ESP32

The task

Button A's ISR puts each press into a FreeRTOS **queue** (`xQueueSendFromISR`). A task takes them out (`xQueueReceive`) and shows the count on the LCD. Template: [GitHub](#).

A ready bounded buffer

A FreeRTOS queue is the bounded buffer of today: `xQueueCreate(10, ...)` makes one with 10 places. A producer and a consumer on the microcontroller.

Check yourself

1. What are the four conditions for a deadlock? Which one does a fixed lock order break?
2. Why did the philosophers' program hang only with the pause? What does it tell you about testing?
3. Why is busy waiting bad? What does `cv.wait(lock, condition)` do with the mutex?
4. When does the producer wait, and when does the consumer wait?
5. What is the difference between a mutex and a semaphore?
6. Why does a pool with more workers than cores not get faster?

Try to answer each question in **2 or 3 sentences**.